HyTMTC: A Pre-Training Method for Multi-Scenario Network Traffic Classification With Hybrid Transformer-Mamba

Yuanming Huang, Xiaojuan Wang, *Member, IEEE*, and Mingshu He, *Member, IEEE*

***Abstract*—Network traffic classification is central to security monitoring and network management in heterogeneous environments. Existing deep learning approaches are often trained for a single scenario and require large amounts of labeled data, making them difficult to reuse when applications, traffic types, or encryption settings change. We present HyTMTC, a pre-training and fine-tuning framework for multi-scenario network traffic classification. HyTMTC encodes each flow as a unified multimodal token sequence that combines raw bytes, packet length, and direction, allowing protocol traces and communication behavior to be modeled together. To match the one-dimensional nature of traffic data, HyTMTC adopts a hybrid Transformer-Mamba backbone. Mamba captures contiguous byte- and packet-level patterns, while Transformer attention strengthens interactions across non-adjacent fields and packets. A self-attention fusion module further integrates the multimodal representations during fine-tuning. This design improves traffic representation without relying on scenario-specific feature engineering. Experiments on seven public datasets show that HyTMTC achieves an average F1-score of 95.03% and outperforms nine representative baselines. It also remains effective in encrypted, VPN, and few-shot settings, and maintains the ability to detect unknown attacks. These results demonstrate its effectiveness and stability for multi-scenario network traffic classification.**

***Index Terms*—Network traffic classification, pre-training, few-shot learning, multi-scenario.**

I. Introduction

With the continuous development of network technologies, network threats also evolve accordingly and become increasingly complex and stealthy. Network traffic classification methods provide an effective way of discovering network threats and play an important role in network security management, such as malicious traffic identification, encrypted traffic analysis in operational networks. Methods based on deep packet inspection (DPI) struggle to adapt to highly complex traffic scenarios, particularly in modern managed networks where encryption and dynamic traffic patterns are widely deployed. Machine learning (ML)-based methods rely on manual feature design, resulting in high feature engineering costs. Traditional deep learning (DL)-based methods can automatically learn representations and behavioral patterns from traffic data, but they typically depend on large-scale, high-quality labeled data, leading to high training and annotation costs. Moreover, they are often designed for a single scenario and exhibit limited adaptability to heterogeneous network environments. As a result, traffic classification models need to be frequently retrained when network conditions or management tasks change, imposing significant computational and operational overhead. In practical network management and security operations, models are expected to be rapidly adapted to new environments with minimal labeled data. The emergence of the pre-training–fine-tuning paradigm alleviates the limitations of traditional DL-based methods and has been widely applied in fields such as natural language processing and computer vision [1], [2], [3], [4]. The pre-training–fine-tuning paradigm requires fewer labeled data and provides improved adaptability, making it suitable for deployment across multiple traffic scenarios in network security management environments.

Most existing pre-training methods for network traffic classification are built on Transformer [5], [6], [7] or Mamba architectures [8]. Transformer-based methods, such as ET-BERT [5] and TrafficFormer [6], are effective in capturing token-level dependencies, but their design is inherited largely from natural language processing. Network traffic sequences are different from natural language: they are shorter, less semantically structured, and dominated by protocol fields, packet boundaries, and transmission behaviors. Using a full Transformer for such sequences may introduce unnecessary model complexity and increase the risk of overfitting, especially when labeled traffic samples are limited. Mamba provides a more compact alternative for modeling one-dimensional sequential traffic. Its selective state-space mechanism is well suited to contiguous byte and packet patterns, such as protocol headers, payload fragments, and packet-wise transmission trends. However, traffic classification also depends on interactions between non-adjacent fields and packets. For example, a byte pattern in one packet may be

Received 7 July 2026; revised 8 August 2026 and 13 August 2026; accepted 17 August 2026. Date of publication 20 August 2026; date of current version 4 September 2026. The work was supported in part by the National Natural Science Foundation of China under Grant 62402053 and Grant 62227805, and in part by the Fundamental Research Funds for the Central Universities under Grant 2025KYQD17(BUPT). The associate editor coordinating the review of this article and approving it for publication was L. Yang. *(Corresponding author: Xiaojuan Wang.)*

Yuanming Huang is with the School of Integrated Circuits, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: huang_yuanming@bupt.edu.cn).

Xiaojuan Wang and Mingshu He are with the School of Cyberspace Security and the Key Laboratory of Trustworthy Distributed Computing and Service, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: wj2718@bupt.edu.cn; hemingshu@bupt.edu.cn).

Digital Object Identifier 10.1109/TNSM.2026.3725912

1932-4537 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

meaningful only when considered together with the packet direction, length, or a related response packet. Pure Mamba models do not explicitly compute pairwise token interactions, which may limit their ability to capture such cross-field and cross-packet dependencies.

Based on these observations, HyTMTC adopts a hybrid Transformer-Mamba architecture. Mamba layers are used to efficiently extract sequential patterns from the one-dimensional traffic representation, while the Transformer layer provides explicit token-to-token interaction for important structural and behavioral cues. This design matches the nature of traffic data: local contiguous patterns are first modeled efficiently, and more global dependencies are then refined through attention. In addition, MoE is introduced to improve the adaptability of the model to heterogeneous traffic scenarios, where different datasets and traffic types may rely on different combinations of byte-level, length-based, and direction-based evidence. Therefore, the architecture is not a simple combination of existing modules, but is designed around the structural properties of network traffic.

Network traffic is structured data and inherently contains semantic information and structural information. Most traditional methods and ML-based methods extract features based on known structural information, indicating that structural information is an important basis for network traffic classification. Meanwhile, complex scenarios commonly employ encryption protocols such as TLS/SSL or communicate through VPN tunnels, resulting in a low amount of plaintext information. In such cases, structural information (including packet length, direction, and header structure) is easier to obtain than semantic information. However, most existing pre-training methods [5], [6], [8], [9] overlook this fact and model traffic solely based on raw byte information. Byte information contains all semantic information and only part of implicit structural information, which limits the model's utilization of structural information. Although PEAN [7] incorporates packet length information during the fine-tuning stage, it still ignores the role of structural information during the pre-training stage.

To address the limitations of existing pre-training methods, we propose HyTMTC. HyTMTC combines Transformer and Mamba architectures to leverage the advantages of both. During sequence learning, HyTMTC enables interactions between arbitrary token pairs and selects important related tokens for each token. Compared with single-architecture models, the effective receptive field [10] (ERF) of HyTMTC is significantly enhanced. The complexity of network traffic is relatively low, and directly using models such as BERT is overly complex. Therefore, HyTMTC employs only three Mamba layers and one Transformer layer, along with a smaller hidden dimension. This design reduces model complexity while ensuring effective classification performance. On the basis of semantic information (byte information), HyTMTC explicitly incorporates the structural information of network traffic. HyTMTC simultaneously injects raw byte information, packet length, and direction information into the sequence and models network traffic based on a multimodal sequence. During the pre-training stage, HyTMTC adopts the Masked Language Modeling (MLM) task [1] to learn general knowledge from the multimodal traffic representation sequence. During the fine-tuning stage, HyTMTC designs a self-attention mechanism to fuse multimodal information in the sequence, further enhancing the model's classification capability in complex scenarios. The code will be released after publication at https://github.com/PolinaH/HyTMTC.

In summary, the contributions of this paper are as follows:

(1) We propose HyTMTC, a pre-training-based method for multi-scenario network traffic classification. HyTMTC effectively integrates the global interaction capability of Transformer with the selective modeling capability of Mamba. The model designed in HyTMTC employs fewer layers and a lower hidden dimension, making it well suited for network traffic data. In addition, HyTMTC designs a self-attention mechanism during the fine-tuning stage, which enhances classification performance in complex scenarios.

(2) Based on the characteristics of network traffic, HyTMTC explicitly injects byte-level and structural information into the model to construct a multimodal traffic sequence. During both the pre-training and fine-tuning stages, HyTMTC is able to learn and represent multimodal traffic knowledge.

(3) HyTMTC achieves effective performance in Internet of Things (IoT) attack detection, encrypted application classification, service/application fingerprinting tasks within VPN tunnels and mobile application identification. Experimental results show that HyTMTC achieves an average F1-score (F1) of 95.03% on seven public datasets, with a coefficient of variation (CV) of 4.88% for the F1. HyTMTC outperforms nine baselines in terms of classification performance and multi-scenario stability. Meanwhile, HyTMTC demonstrates effective classification capability in complex or few-shot scenarios, and maintains detection ability for unknown attacks. Ablation studies demonstrate the advantages of HyTMTC in sequence modeling as well as the ability of the self-attention mechanism to enhance traffic representation in complex scenarios. In addition, HyTMTC exhibits acceptable inference efficiency.

II. Related Work

A. Network Traffic Classification and Representation Learning

Network traffic classification methods can be roughly categorized into traditional methods, ML-based methods, and DL-based methods. Traditional methods [11], [12] mainly rely on port mapping, DPI, and signature matching [13]. They are effective when traffic is plaintext or when ports and signatures are stable, but their performance degrades in encrypted and dynamically changing network environments [14]. ML-based methods [15], [16], [17], [18], [19] usually extract packet-level or flow-level statistical features and then train classifiers such as random forest [20]. FlowPrint [15] is a representative semi-supervised method for mobile-app fingerprinting on encrypted

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

traffic. It builds app fingerprints by using temporal correlations among destination-related traffic features. Although these methods reduce the dependence on payload inspection, they still rely heavily on manually designed features.

DL-based methods [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34] learn traffic representations automatically from raw bytes, packet sequences, traffic images, or graph-structured inputs. CNN-, RNN-, GNN-, and Transformer-based models have been used to capture local byte patterns, temporal packet dependencies, and structural relations between traffic units [14]. Recent work further improves encrypted traffic classification from different perspectives. CLE-TFE [28] performs graph data augmentation on byte-level traffic graphs and uses supervised contrastive learning with cross-level multi-task learning for packet-level and flow-level classification. RBLJAN [30] jointly models byte information and labels to improve robustness. MH-Net [31] constructs multi-view heterogeneous traffic graphs to model byte correlations at different granularities. BPF-GNN [32] further represents encrypted traffic with byte-, packet-, and flow-level graph features. MIETT [35] treats each packet as an instance in a flow-level bag and uses two-level attention to capture token-level and packet-level relationships. Tong et al. [36] improve encrypted traffic classification using a deep domain adaptation network with a smooth characteristic function. Kotak et al. [19] investigate VPN-encrypted network traffic classification using a time-series approach. These studies show that multi-granularity representations, limited-label learning, and adaptability to changing traffic scenarios are important for practical traffic classification.

In recent years, the pre-training–fine-tuning paradigm has been introduced into network traffic analysis [5], [6], [7], [8], [9], [37], [38], [39]. PERT [37] is an early method that applies pre-training to traffic representation learning and constructs dynamic token embeddings at the packet level. ET-BERT [5] introduces Masked BURST Model (MBM) and Same-origin BURST Prediction (SBP) tasks and uses a bidirectional Transformer encoder to learn encrypted traffic representations. PEAN [7] learns dual-view traffic information from raw bytes and packet lengths. YaTC [9] employs a masked autoencoder and multi-level flow representations to extract packet-level and flow-level features. Flow-MAE [38] leverages masked autoencoding for malicious traffic classification. NetMamba [8] extracts hierarchical flow information from raw traffic, converts it into stride sequences, and uses a Mamba-based pre-trained architecture for efficient traffic classification. TrafficFormer [6] designs an efficient pre-trained model for traffic data with MBM and Same Origin-Direction-Flow (SODF) pre-training tasks. MM4flow [39] divides network flows into raw byte streams and transmission patterns and learns multi-modal representations for multiple traffic analysis tasks.

The above methods demonstrate the effectiveness of deep representation learning for traffic classification. However, most of them are still designed for closed-set classification, where all test samples are assumed to belong to known classes. In addition, many methods rely on a single backbone or a single dominant representation form. HyTMTC differs from these studies by injecting raw bytes, packet length, and direction information into a unified sequence representation and by combining Transformer and Mamba modules to balance global dependency modeling and efficient sequence modeling.

B. Unknown and Anomaly Traffic Detection

Beyond closed-set traffic classification, recent studies have investigated unknown and anomaly traffic detection. This direction is important because supervised classifiers may achieve high accuracy on known classes but can still misclassify previously unseen attacks, applications, or abnormal behaviors as known labels. A representative setting is semi-supervised or zero-positive anomaly detection, where the model learns mainly from normal traffic and detects abnormal samples according to reconstruction errors, representation deviations, uncertainty scores, or decision boundaries learned from normal data.

Zheng et al. [40] propose a semi-supervised anomaly network traffic detection framework based on multimodal traffic information fusion. The method extracts low-frequency information from two-dimensional traffic images and performs multimodal fusion for anomaly detection. Lian et al. [41] study payload-level anomaly traffic detection through semi-supervised contrastive learning. MFR [42] adopts a normal-only anomaly detection setting and performs multi-frequency reconstruction over traffic-image representations. FreeUp [43] decouples encrypted traffic images into low- and high-frequency components and dynamically fuses their reconstruction uncertainties for zero-positive anomaly detection. Dang et al. [44] propose a anomaly traffic detection method based on bidirectional normalizing flows, where pseudo-anomaly samples are generated in the latent space. UnDiff [45] uses uncertainty-inspired inter-sample differences to detect anomaly traffic, and ConMD [46] introduces contextual masking distillation for network traffic anomaly detection. These methods directly address the unknown or anomaly detection problem, but their primary goal is usually binary normal/anomalous discrimination or anomaly scoring rather than multi-class traffic classification across heterogeneous datasets.

Open-set and class-incremental traffic detection are also closely related. Reliable open-set network traffic classification [47] focuses on recognizing known traffic while rejecting unknown traffic. Bovenzi et al. [48] systematically benchmark class-incremental learning for DL traffic classification, where new traffic classes are incorporated without training a classifier from scratch. These methods are important for deployment scenarios where new classes appear after training. However, their assumptions and optimization objectives are different from those of supervised multi-class classifiers.

These studies are complementary to HyTMTC. HyTMTC is not designed as a replacement for zero-positive anomaly detectors or open-set rejection models. Its main objective is to learn lightweight and general traffic representations for supervised multi-scenario traffic classification, including IoT malicious traffic classification, VPN-encrypted service/application classification, mobile application identification, and encrypted application fingerprinting. Therefore, in addition to standard

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF HYTMTC WITH REPRESENTATIVE RELATED METHODS

Method	Year	Learning paradigm	Main task	Input/representation	Backbone
FlowPrint [15]	2020	ML-based semi-supervised	Traffic identification	Destination-related flow features	Statistical fingerprinting
Kotak [19]	2025	ML-based time-series	VPN-encrypted traffic classification	Time-series features	Time-series model
CLE-TFE [28]	2024	Supervised graph-based DL	Traffic classification	Byte-level traffic graph	GNN with temporal fusion
MH-Net [31]	2025	Supervised graph-based DL	Traffic classification	Multi-view traffic graph	Heterogeneous GNN
BPF-GNN [32]	2026	Supervised graph-based DL	Encrypted traffic classification	Byte-packet-flow graph	Multi-granularity GNN
Tong [36]	2025	Domain-adaptive DL	Encrypted traffic classification	Flow features	Domain adaptation network
PEAN [7]	2022	Transformer-based pre-training	Traffic classification	Raw bytes+packet length	Transformer
ET-BERT [5]	2022	Transformer-based pre-training	Encrypted traffic classification	Raw-byte sequence	Transformer
YaTC [9]	2023	Transformer-based pre-training	Traffic identification	Traffic representation matrix	Masked autoencoder
TrafficFormer [6]	2025	Transformer-based pre-training	Traffic classification	Bidirectional flow sequence	Transformer
MIETT [35]	2025	Transformer-based pre-training	Encrypted traffic classification	Packet-as-instance bag	Two-level attention Transformer
MM4flow [39]	2025	Transformer-based pre-training	Multi-task traffic analysis	Raw bytes+transmission pattern	Multimodal cross-attention
NetMamba [8]	2024	Mamba-based pre-training	Traffic classification	Stride sequence	Mamba
Bovenzi [48]	2024	Class-incremental learning	Traffic classifier update	Traffic features	Class-incremental algorithms
RoNeTC [47]	2025	Open-set/unknown detection	Open-set traffic classification	Traffic features	Open-set classifier
MFR [42]	2025	Semi-supervised anomaly detection	Anomaly traffic detection	Traffic images	Multi-frequency reconstruction
FreeUp [43]	2026	Zero-positive anomaly detection	Encrypted traffic anomaly detection	Low-/high-frequency traffic images	Frequency-decoupled dual-branch autoencoder
UnDiff [45]	2025	Zero-positive anomaly detection	Anomaly traffic detection	Inter-sample differences	Uncertainty-based detector
ConMD [46]	2026	Zero-positive anomaly detection	Anomaly traffic detection	Traffic contexts	Masking distillation
HyTMTC	2026	Hybrid Transformer-Mamba pre-training	Multi-scenario traffic classification	Raw bytes+length/direction	Hybrid Transformer-Mamba

closed-set evaluation, we further evaluate HyTMTC under an unknown attack detection setting to analyze whether the learned representation can generalize beyond known classes.

C. Hybrid Transformer-Mamba Architecture

Transformer models are effective in modeling global dependencies, but the self-attention mechanism introduces relatively high computational cost for long sequences. Mamba and other state-space models provide an efficient alternative for sequence modeling with linear-time computation [49]. In network traffic analysis, NetMamba [8] demonstrates that a unidirectional Mamba architecture can be adapted to traffic classification with low computational overhead. Regarding scanning strategies, Lian et al. [50] show that multi-view Mamba scanning may introduce redundant representations and propose disentangled fusion to preserve view-specific information, which is consistent with our observation that single-view scanning performs better. However, pure Mamba-based models may not fully capture global interactions among heterogeneous traffic tokens, while pure Transformer-based methods can be less efficient when processing long byte sequences.

Hybrid Transformer-Mamba architectures have recently been explored in several fields. TranMamba [51] and SRMamba-T [52] improve single-image super-resolution. HSI-MFormer [53] applies a hybrid Mamba-Transformer design to hyperspectral image classification. Zhu et al. [54] and Jamba [55] study hybrid architectures for sequence modeling in NLP. ST-MambaSync [56] and MHS-VIT [57] apply related ideas to traffic flow prediction. These studies suggest that Transformer and Mamba can provide complementary modeling capabilities.

Compared with the above studies, the use of a hybrid Transformer-Mamba architecture for network traffic analysis remains insufficiently explored. HyTMTC introduces such a hybrid architecture into traffic representation learning. The Mamba modules are used to efficiently scan token sequences, while the Transformer module strengthens global dependency modeling. This design aims to improve representation capability and maintain lightweight inference for multi-scenario traffic classification.

D. Comparison With Existing Work

Table I summarizes representative studies that are closely related to HyTMTC. These studies cover recent progress in encrypted traffic classification, traffic representation learning, adaptive traffic analysis, pre-trained traffic modeling, and unknown traffic detection. The methods are grouped according to their learning paradigms to clarify their training assumptions and model designs. Their input representations are also summarized in a unified form to make the differences among methods clearer. As shown in the table, HyTMTC differs from existing studies by integrating raw bytes, packet length, and direction into a unified token sequence and by combining Transformer and Mamba modules for multi-scenario traffic classification.

III. THE PROPOSED METHOD

We first explain the research motivation and then outlines the HyTMTC framework, followed by a detailed description of the HyTMTC design, including the network traffic data processing method, the HyTMTC model design, the pre-training procedure, and the fine-tuning procedure.

A. Motivation

Network traffic classification provides fundamental support for various network traffic classification tasks. Compared with methods that rely on features or plaintext classification, approaches that leverage raw bytes and transmission pattern information exhibit broader applicability across different scenarios. Traditional DL-based methods [28], [31] are constrained by the scale and quality of labeled data. The results in Section IV-C show that under few-shot conditions, methods without a pre-training process exhibit significantly degraded classification performance. The pre-training–fine-tuning paradigm effectively alleviates this problem.

To adapt to multi-scenario network traffic, we directly inject raw byte information and structural information into the model. Based on the results in Section IV-F, incorporating structural information on top of raw bytes effectively enhances model capability. Since the input sequence does not contain explicit features, the model's feature extraction and sequence modeling capabilities are particularly important. Transformer

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

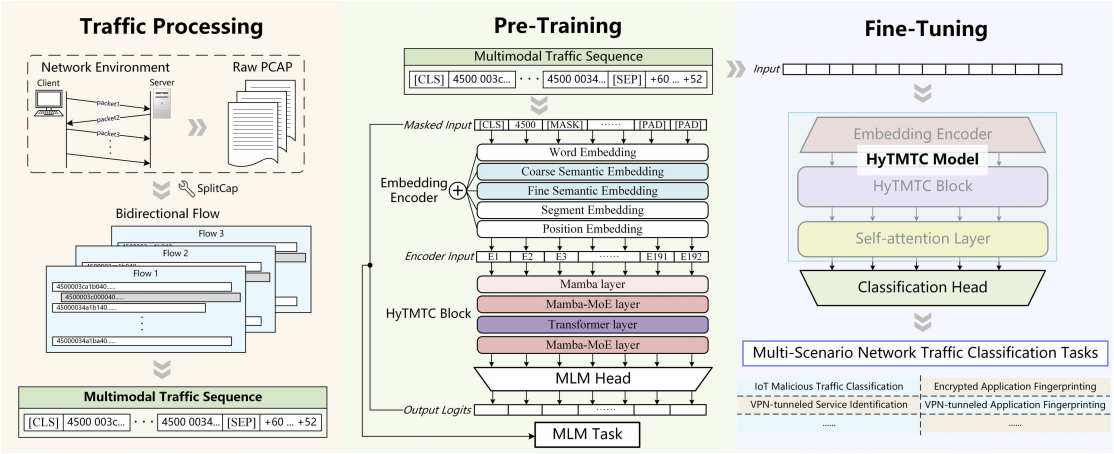


Fig. 1. Overview of HyTMTC. The traffic processing module converts raw PCAP data into a multimodal traffic sequence. The pre-training module applies the MLM task to pre-train the multimodal traffic sequence. The fine-tuning module fine-tunes the pre-trained parameters on different downstream tasks.

assigns attention weights to each token pair and enables complete global modeling. Mamba excels at selecting key token pairs while suppressing information provided by redundant token pairs. However, the strengths of Transformer and Mamba correspond to each other's weaknesses. According to the results in Section IV-E, although Transformer achieves broad dependency coverage, its ability to capture key token pairs is weaker than that of Mamba. Mamba effectively extracts local critical dependencies but lacks sufficient global interaction capability. In the multimodal traffic sequence, the first up to 165 tokens correspond to byte information, while the last up to 20 tokens correspond to structural information. Therefore, the model needs to capture long-range dependencies between byte tokens and structural tokens on the basis of global token interactions. This consideration motivates the hybrid use of Transformer and Mamba for network traffic representation.

B. HyTMTC Overview

As shown in Fig. 1, HyTMTC adopts the pre-training–fine-tuning paradigm, builds the model using the Transformer-Mamba architecture, takes raw network traffic as input, and produces classification results for specific tasks.

HyTMTC first converts raw network traffic into a multimodal traffic sequence and then feeds it into the HyTMTC model to learn structural and semantic information. During the pre-training stage, HyTMTC uses the MLM task to learn general network traffic knowledge in a self-supervised manner on large-scale unlabeled data. In the fine-tuning stage, HyTMTC employs a self-attention mechanism to fuse multimodal information, enabling it to perform network traffic classification tasks across various scenarios. Section III-C describes the processing of raw network traffic, Section III-D details the design of the HyTMTC model, Section III-E presents the pre-training procedure, and Section III-F present the fine-tuning procedure. The overall framework is summarized as follows:

- Raw network traffic processing: Raw network traffic is split into bidirectional flows, and a multimodal traffic sequence is generated based on three modalities: byte modal, packet length modal, and transmission direction modal.
- HyTMTC model design: The HyTMTC model consists of an embedding encoder and HyTMTC blocks. The embedding encoder converts the multimodal traffic sequences into embedding representations. Through five embedding layers, it preserves both semantic information and fine-grained structural information from the original network traffic. The HyTMTC block is designed based on the Transformer-Mamba architecture.
- Pre-training with multimodal data: Pre-training is performed on large-scale unlabeled data using the MLM task. During this stage, the model simultaneously learns multimodal information from each flow.
- Fine-tuning with self-attention mechanism: The model is fine-tuned for downstream tasks based on the pre-trained parameters. In the fine-tuning stage, a self-attention mechanism is designed to fuse multimodal information, thereby improving the model's classification capability in complex scenarios.

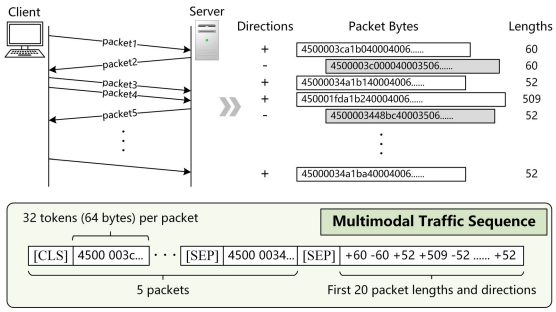


Fig. 2. Raw Network Traffic Processing. The diagram depicts the processing procedure for an individual flow.

C. Raw Network Traffic Processing

We use the SplitCap tool to split raw PCAP files into bidirectional flows, generating multiple flows, each of which serves as a sample. We construct multimodal traffic sequences

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

using both byte-level information and transmission pattern information from the network traffic. Fig. 2 illustrates the raw network traffic processing procedure.

Byte modal: Each bidirectional flow is represented as a sequence of hexadecimal characters. Previous studies have shown that using only the first few packets of a flow is sufficient for effective traffic classification [6]. Therefore, we extract the first five packets from each flow. We randomize the IP addresses and ports within each bidirectional flow in a flow-consistent manner. Specifically, the same original IP address or port in a given bidirectional flow is always mapped to the same randomized value. This operation aims to anonymize IP addresses and ports while preserving the relative transmission relationships among packets. For each packet, we capture 64 bytes starting from the IP layer and generate bi-byte tokens based on two-byte segmentation.

Packet length modal and transmission direction modal: Transmission pattern information, including packet length and direction, plays an important role in analyzing non-plaintext traffic [58]. For example, packet length is effective for encrypted traffic classification, while packet direction is useful for analyzing Tor traffic. We extract the lengths and directions of the first 20 packets in a flow to generate length-direction tokens. In each token, the sign indicates the transmission direction and the value represents the corresponding packet length. The transmission pattern modality includes both packet length and direction, containing up to 20 tokens. These length-direction tokens are concatenated after the byte tokens.

We introduce special tokens [CLS], [SEP], [PAD], and [MASK]. [CLS] marks the beginning of a flow. [SEP] connects the bi-byte tokens of each packet with the length-direction tokens. [PAD] fills the token sequences to match the model's required input length. [MASK] is used for the MLM task.

D. HyTMTC Model Design

The HyTMTC model consists of an embedding encoder and a HyTMTC block. The model takes multimodal traffic sequences as input. The embedding encoder maps the multimodal traffic sequences into embedding representations. The HyTMTC block learns the semantic and structural information of the network traffic from the embedding representations. Compared to the Transformer-based methods, HyTMTC achieves excellent performance on downstream tasks while reducing computational resource requirements. The detailed computation process of the model is described below.

(a) Embedding Encoder

The embedding encoder consists of a word embedding layer, a coarse semantic embedding layer, a fine semantic embedding layer, a segment embedding layer, and a position embedding layer. The input to the embedding encoder is a batch of token sequences $ts \in \mathbb{R}^{BS\times L}$, and the output is the hidden embedding $X_{encoder} \in \mathbb{R}^{BS\times L\times d}$. BS, L and d denote the batch size, the length of a token sequence, and the embedding dimension of both the embedding encoder and the HyTMTC block, respectively. In Section IV, L and d are set to 192 and 256, respectively.

TABLE II

THE INTRA-PACKET SEMANTIC INFORMATION EMBEDDED BY THE COARSE SEMANTIC EMBEDDING LAYER AND THE CORRESPONDING BI-BYTE TOKEN INDICES (IN THE ABSENCE OF OPTION FIELDS) ARE LISTED. THE EMBEDDING INDICES FOR [PAD], [SEP], AND [CLS] ARE 0, 1, AND 2, RESPECTIVELY. THE EMBEDDING INDEX FOR LENGTH-DIRECTION TOKENS IS 8

Layer	Protocol	Embedding Index	Semantics	Token Index
IP	IP	3	IP header	0-9
Transport	TCP	4	TCP header	10-19
	UDP	5	UDP header	10-13
	Other protocol	6	Other protocol header	Determined by the protocol
	-	7	Payload	

TABLE III

THE INTRA-PACKET SEMANTIC INFORMATION EMBEDDED BY THE FINE SEMANTIC EMBEDDING LAYER AND THE CORRESPONDING BI-BYTE TOKEN INDICES (IN THE ABSENCE OF OPTION FIELDS) ARE LISTED. THE EMBEDDING INDICES FOR [PAD], [SEP], AND [CLS] ARE 0, 1, AND 2, RESPECTIVELY. THE EMBEDDING INDEX FOR THE REMAINING BI-BYTE TOKENS IS 14, AND THE EMBEDDING INDEX FOR LENGTH-DIRECTION TOKENS IS 15

Layer	Protocol	Embedding Index	Semantics	Token Index
IP	IP	3	Total length	1
		4	TTL + Protocol	4
		5	Source IP	6-7
		6	Destination IP	8-9
Transport	TCP	7	Source port	10
		8	Destination port	11
		9	Sequence number	12-13
		10	Acknowledgment number	14-15
		11	Data offset + Reserved + TCP flags	16
	UDP	12	Source port	10
		13	Destination port	11

The word embedding layer encodes the token values in the multimodal traffic sequence. The bi-byte tokens in the byte modality range from 0x0000 to 0xFFFF. The length-direction tokens in the packet length and transmission direction modalities range from -1500 to 1500. Tokens are ranked in descending order according to their frequency in the pre-training data, and separate corpora are generated for the bi-byte tokens and length-direction tokens. Finally, the corpora of special tokens, bi-byte tokens, and length-direction tokens are concatenated in order to form the complete word embedding corpus. The complete corpus includes [CLS], [SEP], [PAD], and [MASK].

The coarse semantic embedding layer and the fine semantic embedding layer encode structured semantic information from the IP layer and transport layer headers of packets. The coarse semantic embedding layer performs coarse-grained semantic encoding, distinguishing the IP layer header, TCP header, UDP header, other transport layer protocol headers, and transport layer payload in the multimodal traffic sequence. For the IP layer header and TCP header, the encoding ranges are dynamically determined according to the actual header lengths of the packets. The UDP header is encoded as a fixed-length header, and transport layer protocols other than TCP and UDP are treated as other protocols. The fine semantic embedding layer performs fine-grained semantic encoding, distinguishing 11 types of semantic information within a single packet. Similar to the coarse semantic embedding layer, the encoding ranges of the IP layer header and TCP header are dynamically determined, while the UDP header is encoded as a fixed-length

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

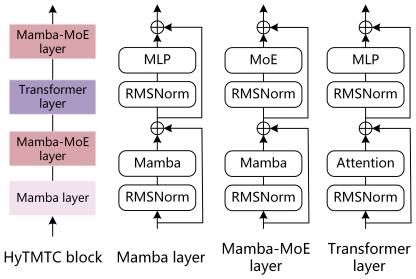


Fig. 3. Structure of the HyTMTC block. $\oplus$ denotes matrix addition.

header. Table II and Table III show the actual encoding values in the absence of option fields. In the actual implementation, the length of the IP layer header is determined by the IHL field, and the length of the TCP header is determined by the Data Offset field.

The segment embedding layer and the position embedding layer provide the model with packet-level temporal information and the overall structure of a flow. The segment embedding layer encodes the order of packets in the multimodal traffic sequence, providing the model with temporal relationships between packets. Tokens belonging to the same packet share the same segment index, which increments according to the packet order. The length-direction tokens are treated as an 'additional packet' in this encoding. The position embedding layer encodes the position of each token in the multimodal traffic sequence, providing the model with structural relationship among tokens. The position indices increment according to the token order.

Let a token sequence be processed by the word embedding layer, the coarse semantic embedding layer, the fine semantic embedding layer, the segment embedding layer, and the position embedding layer, producing embeddings Emb_w, Emb_c, Emb_f, Emb_s, and Emb_p, respectively. The complete embedding output of the Embedding encoder is $X_{encoder}$:

$$X_{encoder} = Emb_w + Emb_c + Emb_f + Emb_s + Emb_p \quad (1)$$

(b) HyTMTC Block

The structure of the HyTMTC block is shown in Fig. 3. It consists of one Mamba layer, two Mamba-MoE layer, and one Transformer layer.

The HyTMTC block first learns long-range dependencies within sequences via the Mamba layer, such as relationships between different packet headers. Subsequently, it calculates self-attention scores of global tokens through the Transformer layer. Therefore, the HyTMTC block simultaneously learns both long-range dependencies and accurate global dependencies within sequences. Additionally, the HyTMTC block computes the self-attention score only once, which reduces the impact of key-value cache size for model efficiency. The MoE component enables the HyTMTC Block to dynamically select different expert networks based on varying tokens or tasks. This enhances the model's ability to process complex data and improves its stability across multiple tasks. The number of expert networks within the MoE component is set to 8, each toke is able to select its expert network from 2 expert networks. Parameters are shared among layers of the same type to accelerate model training and inference. Both the input X_{in} and the output X_{out} of the HyTMTC block belong to $\mathbb{R}^{BS\times L\times d}$, and X_{out} undergoes different subsequent processing during the pre-training and fine-tuning stages.

The HyTMTC block first learns long-range dependencies within sequences via the Mamba layer, such as relationships between different packet headers. Subsequently, it calculates self-attention scores of global tokens through the Transformer layer. Therefore, the HyTMTC block simultaneously learns both long-range dependencies and accurate global dependencies within sequences. Additionally, the HyTMTC block computes the self-attention score only once, which reduces the impact of key-value cache size for model efficiency. The MoE component enables the HyTMTC Block to dynamically select different expert networks based on varying tokens or tasks. This enhances the model's ability to process complex data and improves its stability across multiple tasks. The number of expert networks within the MoE component is set to 8, each toke is able to select its expert network from 2 expert networks. Parameters are shared among the same layers. Specifically, in the HyTMTC block, the second and fourth Mamba-MoE layers refer to the same module object and share the same learnable parameters. The architectural variants compared in Section IV-E also adopt parameter sharing. For example, the four layers of the Transformer variant share the same set of learnable parameters. The experimental results show that the parameter-sharing strategy preserves model performance while reducing the number of parameters. Both the input X_in and the output X_out of the HyTMTC block belong to $\mathbb{R}^{BS\times L\times d}$, and X_out undergoes different subsequent processing during the pre-training and fine-tuning stages.

The Mamba layer consists of an RMSNorm [59] module, a standard unidirectional Mamba module, and an MLP module.

Given the hidden embedding $x \in \mathbb{R}^{BS\times L\times d}$, the computation is as follows. First, x is sequentially passed through the RMSNorm module and the Mamba module to obtain an intermediate variable x':

$$x' = \mathrm{Mamba}(\mathrm{RMSNorm}(x)) \quad (2)$$

and a residual connection is added to the input x to produce another intermediate variable z,

$$z = x + x' \quad (3)$$

then, z is passed through the RMSNorm module and the MLP module sequentially to obtain the output y, which is computed as:

$$y = \mathrm{MLP}(\mathrm{RMSNorm}(z)) + z \quad (4)$$

The MLP module consists of two linear layers with hidden dimensions $256 \to 1024$ and $1024 \to 256$, respectively.

Fig. 4 shows the structure of the Mamba module. The input to the Mamba module has shape $\mathbb{R}^{BS\times L\times d}$ and is first mapped by linear projection1 to a tensor of shape $\mathbb{R}^{BS\times L\times 2d}$. This tensor is then split along the last dimension into two tensors of shape $\mathbb{R}^{BS\times L\times d}$. These two tensors are processed separately, then multiplied element-wise, and finally mapped by linear projection2 to produce the output of the module. $||$ denotes tensor splitting, σ denotes the SiLU activation function, and

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

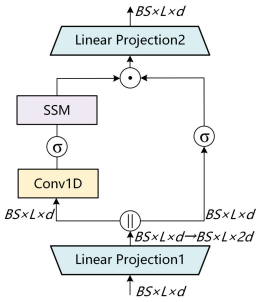


Fig. 4. Structure of the Mamba module. || denotes tensor splitting, σ denotes the SiLU activation function, and $\odot$ denotes element-wise multiplication.

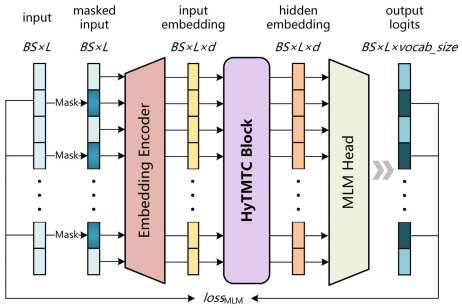


Fig. 5. Pre-training process of HyTMTC.

$\odot$ denotes element-wise multiplication. In the Mamba module, the selective scan operates along the token dimension in a single direction, from the first token to the last token.

State Space Models (SSMs) are a class of sequence models that represent the evolution of hidden states using linear dynamical systems with input-driven updates. Unlike attention-based models, which rely on pairwise interactions, SSMs capture long-range dependencies by parameterizing state transitions and input/output mappings in a recurrent yet highly parallelizable manner. The Mamba module in the HyTMTC model employs a diagonal structured SSM, in which the transition matrix A is diagonal and the dynamic parameters Δ, B, and C are generated from the input.

Given the input $x \in \mathbb{R}^{BS\times L\times d}$ from the Conv1D, the SSM first generates the dynamic parameters Δ, B, and C via a linear projection. The step size Δ is subsequently mapped to the hidden dimension, resulting in $\Delta' \in \mathbb{R}^{BS\times L\times d}$.

At each time step t, the hidden state $x_t \in \mathbb{R}^{B\times d\times n}$ is updated as:

$$x_t = e^{\Delta'_t A} \odot x_{t-1} + \Delta'_t \cdot (B_t \odot u_t) \tag{5}$$

where $A \in \mathbb{R}^{d\times d_{state}}$ is a diagonal transition matrix, d_{state} denotes the number of internal states per feature channel, and $\odot$ denotes element-wise multiplication. The output is then obtained by:

$$y_t = C_t^\top x_t + D \odot u_t \tag{6}$$

where $D \in \mathbb{R}^d$ represents a skip connection term. Finally, the outputs $y_t{}_{t=1}^{L}$ are stacked along the temporal dimension to form the SSM output $y \in \mathbb{R}^{BS\times L\times d}$, which is subsequently fed into the gating mechanism and the linear projection2.

The Mamba-MoE layer replaces the original MLP module in the Mamba layer with a MoE module while keeping the other modules unchanged. Given the hidden embedding $x \in \mathbb{R}^{BS\times L\times d}$, the same intermediate variable z is obtained following the steps in the Mamba layer. The output y is computed as:

$$z_{norm} = \mathrm{RMSNorm}(z) \tag{7}$$

$$y = \sum_{i\in\mathcal{S}(z)} g_i(z_{norm}) \cdot \mathrm{MLP}_i(z_{norm}) + z \tag{8}$$

where $\mathrm{MLP}_i(\cdot)$ denotes the i-th expert network, implemented as a single linear layer in this design. $g_i(z)$ is the softmax-normalized routing weight for expert i among all E experts. The set $S(z) \subseteq \{1, 2, \ldots, E\}$ contains the indices of the top-k experts selected for each token, and the gating weights over $S(z)$ are normalized to sum to 1. The total number of experts E is set to 8, and the top-k value is set to 2.

The Transformer layer has been extensively applied across various domains, so its detailed computations are omitted here. The primary difference from the Mamba layer lies in the replacement of the Mamba module with a multi-head self-attention module. In the Transformer layer, the input token sequence length and embedding dimension remain the same as in the Mamba layer, while the hidden size and feedforward size are set to 256 and 1024, respectively.

E. Pre-Training With Multimodal Data

Pre-training enables models to learn generic patterns from data [60]. HyTMTC adopts pre-training to acquire generic knowledge of network traffic in a self-supervised manner. Since network traffic data shares similarities with textual data, we employ the MLM task to pre-train the model. The MLM task enables the model to predict the values of masked tokens based on contextual information, fully leveraging the information within sequences. Meanwhile, the random masking strategy allows the model to gain a deep understanding of sequence structure and semantics, better learning inter-packet relationships, long-range dependencies, etc. Fig. 5 illustrates the pre-training process.

The principle of the MLM task is to randomly mask a subset of tokens in the multimodal traffic sequence and predict only the masked tokens, enabling the model to learn bidirectional contextual information. We adopt the same masking strategy as BERT: 15% of the tokens are randomly selected, of which 80% are replaced with [MASK], 10% are replaced with other tokens, and the remaining 10% remain unchanged. The MLM task computes the cross-entropy loss over the masked tokens. Let $\mathcal{S}$ denote the set of masked token positions and y_i denote the ground-truth token at position i. The pre-training loss $loss_{\mathrm{MLM}}$ is defined as:

$$loss_{\mathrm{MLM}} = -\frac{1}{|\mathcal{S}|}\sum_{i\in\mathcal{S}} \log p_\theta(y_i) \tag{9}$$

where $p_\theta(y_i)$ is the predicted probability of y_i from the model's softmax output at position i, and θ represents the trainable parameters of HyTMTC.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

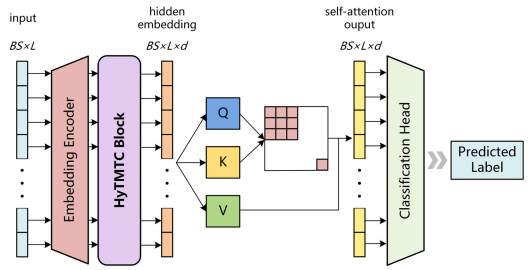


Fig. 6. Fine-tuning process of HyTMTC.

F. Fine-Tuning With Self-Attention Mechanism

After pre-training, the HyTMTC model has already acquired general knowledge of network traffic. The pre-trained parameters are first loaded into HyTMTC, and the model is then fine-tuned in a supervised manner using network traffic from specific scenarios. Since the input samples of HyTMTC are multimodal, a multi-head self-attention mechanism is introduced during fine-tuning. This mechanism adaptively reweights token importance according to the specific requirements of each downstream task. Fig. 6 illustrates the fine-tuning process.

Given the hidden embedding $x \in \mathbb{R}^{BS\times L\times d}$, the query, key, and value matrices for the i-th head are obtained through linear projections:

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V, \quad i = 1, \ldots, n_{head} \tag{10}$$

where W_i^Q, W_i^K and $W_i^V \in \mathbb{R}^{d\times d_h}$, and $d_h = d/n_{head}$. The attention output of each head is computed as:

$$\text{head}_i = \text{Softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}}\right) V_i \tag{11}$$

The outputs from all heads are concatenated and linearly transformed to produce the final output:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{n_{head}})W^O \tag{12}$$

where $W^O \in \mathbb{R}^{d\times d}$ is a trainable weight matrix.

After the self-attention mechanism, the hidden embedding passes through two linear layers to predict the labels. The fine-tuning loss $loss_{ft}$ is defined as:

$$loss_{ft} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{M} y_{n,i}\log P_{n,i} \tag{13}$$

where N denotes the number of input samples, M denotes the number of classes, $y_{n,i} \in \{0, 1\}$ indicates whether the n-th sample belongs to the i-th class, and $P_{n,i}$ represents the predicted probability of the n-th sample belonging to the i-th class.

After fine-tuning, HyTMTC can perform multiple downstream tasks. The downstream tasks implemented in this work include IoT malicious traffic classification, encrypted application fingerprinting, VPN-tunneled service identification, and VPN-tunneled application fingerprinting.

TABLE IV
SELECTION RATIOS AND SAMPLE COUNTS OF THE PRE-TRAINING DATASETS

Dataset	Ratio	Number of Samples
ISCX-NonVPN [61]	20%	28869
CICAndMal2017 [62]	32%	67917
Browser [15]	48%	60000
CICIOT2023 [63]	100%	56967
Total		213753

IV. EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of HyTMTC through multiple experiments.

A. Experimental Setup

1) Dataset: In the pre-training stage, HyTMTC uses four datasets: ISCX-NonVPN [61], CICAndMal2017 [62], Browser [15], and CICIOT2023 [63]. The details of the pre-training datasets are as follows.

- ISCX-NonVPN [61] is released by Draper-Gil et al. and consists of the non-VPN portion of the complete ISCX-VPN-NonVPN dataset [61]. The traffic includes Email, Chat, Streaming, File Transfer, VoIP, and P2P, covering common network services and applications. The dataset size is 6.2 GB.
- CICAndMal2017 [62] is released by Lashkari et al. and contains benign and malware samples collected from multiple sources. We use all benign samples and randomly select malware samples equal to 10% of the benign data. The dataset size is 6.4 GB.
- Browser [15] is released by Van et al. and contains traffic fingerprints from 1,000 websites collected via Android smartphones. The dataset size is 7.0 GB.
- CICIOT2023 [63] is released by Neto et al. and contains both normal traffic and various attack traffic collected from real IoT networks. Attack types include DDoS, Brute Force, and Spoofing, among others. We use all normal traffic and randomly select attack traffic equal to 10% of the normal data. The dataset size is 2.1 GB.

To accelerate pre-training, only a subset of the generated pre-training samples is used. The selection ratios and sample counts are listed in Table IV.

In the fine-tuning stage, we use seven datasets collected from IoT scenarios, encrypted scenarios, VPN tunnel scenarios, and real mobile network scenarios. We randomly selected 500 samples from each class and split them into training, validation, and test sets at a ratio of 8:1:1. The details of the fine-tuning datasets are as follows.

- MedBIoT [64] is released by Guerra-Manzanares et al. and contains normal traffic and actual botnet malicious traffic collected from a medium-sized IoT network comprising 83 IoT devices. This dataset is used for the IoT malicious traffic classification task.
- CICIoMT2024 [65] is released by Dadkhah et al. and contains normal traffic and multiple attack types collected from an Internet of Medical Things (IoMT) network with 25 real devices and 15 simulated devices. This dataset is used for the IoT malicious traffic classification task.
- USTC-TFC [66] is released by Wang et al. and contains encrypted traffic from various benign and malicious

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

applications. We select traffic from 9 benign applications and 9 malicious applications. This dataset is used for the encrypted application fingerprinting task.

- ISCX-VPN-service [61] is released by Draper-Gil and consists of traffic transmitted through VPN tunnels from the complete ISCX-VPN-NonVPN dataset [61]. The traffic is divided into six classes according to service type.
- The ISCX-VPN-application [61] comes from the same source as the ISCX-VPN-service. The traffic is divided into 14 classes according to application type.
- NUDT [67] is a mobile network traffic dataset released by the Network Forensics Research Lab. Based on the traffic capture software they developed, network flows are accurately labeled and collected. We select the 20 applications with the largest traffic proportions. This dataset is used for mobile application identification.
- CSTNET [68] is collected on CSTNET from Alexa Top5000 websites deployed with TLS 1.3. We select the 19 applications with the largest traffic proportions. This dataset is used for encrypted application fingerprinting.

For each dataset, deduplication is performed twice using SHA-256: (1) duplicate bidirectional flow samples are removed after the flows are split with SplitCap; and (2) a second round of deduplication is applied after the final samples are generated. The training/validation/test split is then performed at an 8:1:1 ratio after deduplication, while ensuring that flows with the same five-tuple are not distributed across the training, validation, and test sets. When device information is available, traffic from the same device is kept entirely within either the training, validation, or test set. This group-level separation further reduces the risk of data leakage across different subsets. For the ISCX datasets, the NonVPN subset is used for pre-training and the VPN subset for fine-tuning, with no overlap between the two stages.

2) Evaluation Metrics: We adopt four widely used metrics to evaluate the experiments: accuracy (Acc), precision (Pre), recall (Rec), and F1. The metrics are computed based on True Positive (TP), True Negative (TN), False Negative (FN), and False Positive (FP) values. All the metrics are calculated using the macro averaging method.

3) Implementation Details: Pre-training experiments are conducted based on Intel(R) Xeon(R) Silver 4310 CPU@2.10GHz and 2 NVIDIA GeForce RTX 4090 GPUs. The batch size is set to 128 (the effective batch size is 256). The total number of training steps is set to 42,000. The learning rate is set to 0.0006 with a warmup ratio of 0.1 and a cosine decay scheduling strategy. The optimization algorithm used for the model is the ADAM optimizer [69].

Fine-tuning experiments are conducted based on Intel(R) Xeon(R) Silver 4310 CPU@2.10GHz and a NVIDIA GeForce RTX 4090 GPU. The batch size is set to 128. The total number of epochs is set to 50. The learning rate is set to 0.00025 with a warmup ratio of 0.1 and a linear decay scheduling strategy. The optimization algorithm used for the model is the ADAM optimizer [69]. Early stopping is used for HyTMTC, and training is stopped if the F1 does not improve for 10 consecutive epochs.

4) Baselines: We implement nine baselines. The pre-training-based baselines are trained on the same pre-training dataset and with the same data scale as HyTMTC. For a fair comparison, all baselines take the bidirectional-flow PCAPs produced by SplitCap in our pipeline as input samples. Thus, CLE-TFE, MH-Net, and ET-BERT are evaluated with flow-level inputs rather than packet-level inputs.

For the DL-based and pre-training-based baselines, we extract either the first 64 bytes of the first five packets starting from the IP layer, or the direction and length information of the first 20 packets, depending on the input format required by each method. We then construct the corresponding datasets following the data processing procedure of each baseline. The pre-training-based baselines use the same pre-training data as HyTMTC, and their total number of processed samples, measured by the product of training steps and batch size, is kept consistent with HyTMTC. All baselines are fine-tuned on the same datasets and with the same sample sizes as HyTMTC. For the DL-based and pre-training-based baselines, the learning rate, number of epochs, and other hyperparameters follow the settings reported in the original papers or released code. Early stopping is used for these baselines, and training is stopped if the F1 does not improve for 10 consecutive epochs.

ML-based methods: FlowPrint [15] performs traffic identification based on statistical features in a semi-supervised manner.

DL-based methods: CLE-TFE [28] constructs a GNN, converts raw bytes into traffic graphs, and trains the model using supervised contrastive learning for traffic identification. It uses flow-level data as input. MH-Net [31] builds a heterogeneous GNN, leverages heterogeneous features of traffic graphs, and enhances traffic representation using multi-task contrastive learning. It also uses flow-level data as input.

Pre-training–fine-tuning based DL methods with Transformer architecture: PEAN [7] learns dual-view traffic information from raw bytes and packet lengths. ET-BERT [5] converts raw bytes into traffic samples and uses two pre-training tasks, MBM and SBP. It uses flow-level data as input. YaTC [9] converts raw bytes into traffic representation matrices for traffic identification. TrafficFormer [6] converts raw bytes into bidirectional flow samples, uses MBM and Same Origin-Direction-Flow (SODF) as pre-training tasks, and introduces array augmentation during fine-tuning. MM4flow [39] divides network flows into raw byte streams and transmission patterns, and fuses the two modalities with cross-attention for downstream tasks.

Pre-training–fine-tuning based DL methods with Mamba architecture: NetMamba [8] converts raw bytes into stride sequences and completes pre-training and fine-tuning based on a state-space model.

The pre-training methods were all pre-trained using the same dataset and scale as HyTMTC. All baselines were tuned using their optimal hyperparameters.

B. Performance Comparison Between HyTMTC and Baselines

The results in Table V show that HyTMTC achieves the best performance on four datasets, with an average F1 of

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

TABLE V
THE MULTI-CLASS CLASSIFICATION PERFORMANCE OF HYTMTC AND THE BASELINES ON FIVE DATASETS

Method	Dataset							Average F1	F1 Stability (CV)
	MedBIoT	CICIoMT2024	USTC-TFC	ISCX-VPN -service	ISCX-VPN -application	NUDT	CSTNET		
FlowPrint [15]	0.3765	0.3285	0.1912	0.5168	0.4595	0.3160	0.0437	0.3189	0.5031
CLE-TFE [28]	0.8255	0.6923	0.9846	0.9054	0.8194	0.8505	0.8580	0.8480	0.1051
MH-Net [31]	0.9331	0.8406	0.9757	0.9282	0.7753	0.8413	0.7403	0.8621	0.1011
PEAN [7]	**1.0000**	0.5370	0.9739	0.9392	0.6522	0.8044	0.9437	0.8358	0.2142
ET-BERT [5]	0.9607	0.8656	0.9719	0.8708	0.7316	0.7178	0.8740	0.8561	0.1164
YaTC [9]	0.9747	0.9177	0.9928	0.9314	0.7139	0.9282	0.9482	0.9153	0.1013
TrafficFormer [6]	0.9338	0.8621	0.9840	0.8835	0.7357	0.7094	0.9042	0.8590	0.1178
NetMamba [8]	0.9142	0.7784	0.9041	0.9379	**0.8758**	0.6581	0.8097	0.8397	0.1174
MM4flow [39]	-	**0.9366**	0.9489	0.9621	0.7664	0.8688	0.9705	0.9089	0.0865
HyTMTC	0.9782	0.9260	**0.9961**	**0.9855**	0.8645	**0.9501**	**0.9714**	**0.9503**	**0.0488**

95.03%, which is 3.5% higher than the best baseline (YaTC). This result indicates that HyTMTC has better multi-scenario generalization capability. The CV of the F1 obtained by HyTMTC across different datasets is 4.88%, which is better than those of the baselines, indicating that HyTMTC shows better stability across different traffic scenarios.

1) Analysis of Results on Each Dataset: (1) The MedBIoT and CICIoMT2024 datasets are both collected in IoT scenarios. On MedBIoT, HyTMTC and most baselines achieve relatively high F1, mainly because this dataset contains only three physical devices, limited device types, and a small number of relatively simple attack patterns. Therefore, the traffic behaviors in MedBIoT are easier to distinguish. In contrast, CICIoMT2024 is collected from a more realistic and complex IoT scenario with 25 physical devices, full-lifecycle normal traffic, and diverse protocol-specific attacks. As a result, PEAN, NetMamba, and CLE-TFE show F1 drops of more than 10% on CICIoMT2024 compared with MedBIoT, indicating that these baselines have limited robustness in complex IoT scenarios. HyTMTC shows only about a 5% performance drop, demonstrating its stronger adaptability to complex traffic environments.

(2) For the USTC-TFC dataset, all methods except FlowPrint achieve F1 above 90%. Common protocols such as HTTPS and FTPS are mainly used in the collection scenario of this dataset, and more complex encryption methods such as VPN are not involved. The traffic behavior characteristics of different applications can still be clearly distinguished after encryption, and some application layer data are not encrypted. Therefore, the traffic behavior patterns in this dataset are easy to learn.

(3) The ISCX-VPN-service and ISCX-VPN-application datasets are both collected from VPN-encrypted tunnels. The service types and application types are diverse, and few effective features can be extracted. The F1 of HyTMTC is only about 1% lower than that of the best baseline NetMamba. This result indicates that HyTMTC still has the ability to represent the behavior patterns of different services and applications in complex scenarios with encrypted obfuscation.

(4) The NUDT dataset is used for mobile application classification. On this dataset, the performance gap among different methods is large. In mobile traffic scenarios, application-layer content is often protected by encryption, and different applications may share similar services, SDKs, or communication protocols. As a result, application-specific features become difficult to distinguish. HyTMTC achieves the best F1 of 93.01%, indicating that it can effectively capture discriminative behavioral patterns from encrypted mobile application traffic.

(5) The CSTNET dataset is used for encrypted application fingerprinting under TLS 1.3. On this dataset, most deep representation methods achieve relatively high F1, suggesting that TLS 1.3 encryption does not completely eliminate application-specific traffic patterns. Features such as early-flow byte distributions and transmission behaviors remain learnable. HyTMTC achieves the highest F1 of 97.14%, slightly outperforming MM4flow, demonstrating its ability to learn robust application representations under modern encrypted traffic scenarios.

2) Comparison Between HyTMTC and MM4FLOW: Among the pre-training-based methods, both HyTMTC and MM4flow use multi-modal traffic representations, but they differ in how different modalities are organized and learned. MM4flow models byte information and transmission patterns as two separate modalities, where BERT-bytes and BERT-ps are pre-trained independently and then fused during fine-tuning. In contrast, HyTMTC jointly encodes packet bytes and the corresponding transmission direction information within the same traffic sample. This unified representation preserves the packet-level correspondence between encrypted byte patterns and communication behaviors, allowing the model to learn not only what byte patterns appear, but also in which direction and at which stage of the flow they appear. This is important for encrypted traffic classification, because application-layer content is partially hidden, and discriminative cues often come from the joint distribution of early packet bytes, directions, and request-response behaviors. Therefore, compared with the late fusion strategy of MM4flow, HyTMTC can learn cross-modal dependencies more directly from the input sequence. On the six datasets where MM4flow reports results, HyTMTC achieves higher F1 on five datasets and improves the average F1 from 90.89% to 94.56%, especially on ISCX-VPN-application and NUDT. These results indicate that the unified multi-modal representation of HyTMTC is more effective than separately modeling and later fusing different modalities.

3) Analysis of the Effectiveness of HyTMTC in Encrypted Scenarios: HyTMTC remains effective on encrypted traffic because it does not depend on plaintext application semantics. In encrypted flows, including TLS and VPN traffic, application messages, commands, file contents, and other payload semantics are hidden, but the flow still exposes struc-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

THE F1 OF HYTMTC AND THE BASELINES WHEN THE NUMBER OF TRAINING SAMPLES PER CLASS IS 30 AND 50. THE VALUES IN PARENTHESES INDICATE THE DIFFERENCE FROM THE F1 OBTAINED USING THE FULL TRAINING SET

Method	CICIoMT2024		ISCX-VPN-service	
	size=30	size=50	size=30	size=50
FlowPrint [15]	0.1875(-0.1410)	0.2669(-0.0616)	0.4644(-0.0524)	0.6336(+0.1168)
CLE-TFE [28]	0.8162(+0.1239)	0.8499(+0.1576)	0.6366(-0.2688)	0.2947(-0.6107)
MH-Net [31]	0.6364(-0.2042)	0.6364(-0.2042)	0.5968(-0.3314)	0.7304(-0.1978)
PEAN [7]	0.5370(-0.0000)	0.5513(+0.0143)	0.6440(-0.2952)	0.8047(-0.1345)
ET-BERT [5]	0.7909(-0.0747)	0.8516(-0.0140)	0.5897(-0.2811)	0.7104(-0.1604)
YaTC [9]	0.8339(-0.0838)	0.9177(-0.0000)	0.7794(-0.1520)	0.8175(-0.1139)
TrafficFormer [6]	0.8606(-0.0015)	0.8568(-0.0053)	0.7750(-0.1085)	0.7751(-0.1084)
NetMamba [8]	0.7130(-0.0654)	0.7345(-0.0439)	0.7366(-0.2013)	0.8385(-0.0994)
MM4flow [39]	0.8187(-0.1179)	0.8417(-0.0949)	0.7849(-0.1772)	0.8846(-0.0775)
HyTMTC	**0.9077**(-0.0183)	**0.9278**(+0.0018)	**0.8489**(-0.1366)	**0.8927**(-0.0928)

tural traces. Packet order, packet size, transmission direction, transport-layer behavior, and part of the early protocol framing remain observable. More importantly, these features should be interpreted in combination. A byte pattern observed in a client-to-server packet may reflect request or handshake behavior, whereas a similar pattern in the reverse direction may correspond to the response side. Similarly, an encrypted byte fragment may have different meanings when it appears in a small control packet and when it appears in a large data-carrying packet. HyTMTC keeps these packet-level associations by encoding bi-byte tokens, packet length, and direction in the same traffic sequence. Thus, it classifies encrypted flows by modeling the relationships among byte-level protocol structures, packet-size distributions, and request-response behaviors, rather than by relying on recoverable plaintext content.

The ablation results also support this explanation. Direction alone achieves only 50.94% average F1, which shows that communication orientation provides limited class-specific information. Length alone reaches 80.07%, indicating that packet-size patterns are useful but insufficient for fine-grained classification. Bi-byte tokens achieve 90.55%, suggesting that byte-level regularities, including header fields, protocol formats, and early-flow structures, provide stronger discriminative cues. The full HyTMTC reaches 95.01% average F1 and outperforms all single- and two-modality variants. This result shows that its advantage comes from modeling the association between byte patterns, packet sizes, and transmission directions. Although the ablation datasets include both encrypted and non-encrypted scenarios, this association is particularly important for encrypted traffic, where application-layer semantics are unavailable and the remaining discriminative information is mainly reflected in the interaction between byte-level and behavioral features.

C. Few-Shot Performance Comparison Between HyTMTC and Baselines

We randomly select 30 and 50 samples per class from the training set to construct few-shot training sets, while keeping the validation and test sets unchanged. As shown in Table VI, HyTMTC achieves the highest F1 on both CICIoMT2024 and ISCX-VPN-service, suggesting that it is less affected by the shortage of labeled samples.

This advantage can be attributed to the way HyTMTC learns and uses traffic representations. Instead of relying entirely on the few labeled flows in the downstream task, HyTMTC starts from representations learned through pre-training on unlabeled traffic. These representations already encode recurring patterns in packet bytes, packet structures, and cross-packet dependencies. As a result, few-shot fine-tuning mainly adjusts the learned representation to the target labels, rather than learning discriminative traffic features from scratch. This is important for traffic classification, where byte sequences are sparse and highly variable, and purely supervised models can easily overfit when only a few samples are available.

HyTMTC also benefits from combining raw bytes with packet length and direction. Raw bytes retain local protocol and application traces, while length and direction describe the communication behavior of a flow, such as request-response patterns and transmission asymmetry. These signals are especially useful for encrypted and VPN traffic, where payload semantics are hidden but structural and behavioral patterns remain observable. By jointly modeling these complementary cues, HyTMTC can obtain more stable class representations under limited supervision.

TABLE VII

PERFORMANCE COMPARISON ON UNKNOWN ATTACK DETECTION USING THE CICIOMT2024 DATASET. EACH ATTACK CATEGORY IS EXCLUDED FROM THE TRAINING SET IN TURN AND USED ONLY FOR UNKNOWN TESTING. THE MODELS ARE TRAINED AS BINARY CLASSIFIERS USING BENIGN AND KNOWN ATTACK SAMPLES

Unknown attack	Method	Known			Unknown
		Pre	Rec	F1	Rec
Category 1 DDoS	MM4flow [39]	0.9412	0.9371	0.9390	0.5800
	YaTC [9]	0.9306	0.9306	0.9265	0.9000
	HyTMTC	0.9500	0.9643	0.9552	**0.9500**
Category 2 DoS	MM4flow [39]	0.8684	0.8694	0.8683	0.7600
	YaTC [9]	0.9379	0.9414	0.9395	0.9400
	HyTMTC	0.9762	0.9722	0.9735	**0.9444**
Category 3 Spoofing	MM4flow [39]	1.0000	1.0000	1.0000	0.0500
	YaTC [9]	0.9500	0.9643	0.9552	0.1000
	HyTMTC	1.0000	1.0000	1.0000	**0.3600**

D. Performance Comparison Between HyTMTC and Baselines on Unknown Attack Detection

Table VII reports the results when one attack category in CICIoMT2024 is excluded from training and used only for unknown testing. The model is trained as a binary classifier using benign and attack samples, and unknown recall is used to measure whether unseen attacks can still be detected as malicious traffic. HyTMTC achieves the highest unknown recall in all three settings. When DDoS and DoS are used as unknown attacks, HyTMTC reaches recall scores of 95% and 94.44%, respectively. This indicates that the model can learn attack-related traffic patterns that are not limited to the specific attack categories observed during training. The Spoofing case is more difficult. HyTMTC obtains an unknown recall of 36%, which is still not satisfactory, but it is higher than those of MM4flow and YaTC, whose recall scores are only 5% and 10%. This shows that Spoofing is hard to detect in this setting, while HyTMTC still transfers the learned attack representation more effectively than the compared methods.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII

COMPARISON OF CLASSIFICATION PERFORMANCE BETWEEN HYTMTC AND SINGLE-ARCHITECTURE MODELS

Method	Dataset					Average F1
	MedBIoT	CICIoMT 2024	USTC -TFC	ISCX-VPN -service	ISCX-VPN -application	
Mamba	0.9424	0.8524	0.9784	0.9256	0.6899	0.8777
Mamba+MoE	**0.9782**	0.9001	0.9891	0.9698	0.7405	0.9155
Transformer	0.9592	0.6475	0.9885	0.9503	0.8053	0.8701
Transformer+MoE	**0.9782**	0.9212	**0.9978**	0.9767	0.8035	0.9355
HyTMTC	**0.9782**	**0.9260**	0.9923	**0.9800**	**0.8733**	**0.9499**

TABLE IX

ABLATION RESULTS OF UNIDIRECTIONAL AND BIDIRECTIONAL MAMBA VARIANTS ACROSS DIFFERENT DATASETS

Method	Dataset					Average F1
	MedBIoT	CICIoMT 2024	USTC -TFC	ISCX-VPN -service	ISCX-VPN -application	
Bidirectional Mamba	0.9782	0.9091	0.9888	0.9740	0.8513	0.9403
Unidirectional Mamba	0.9782	0.9260	0.9923	0.9800	0.8733	**0.9499**

E. Performance and ERF Comparison Across Different Architectures

In Table VIII, the first four variants are constructed by stacking four layers of the specified architecture, matching the number of architecture layers in HyTMTC. For this experiment, the training set contains 300 samples per class, while the validation and test sets remain unchanged. All models are trained for 10 epochs. The variants using only Mamba or only Transformer obtain an average F1 about 7% lower than HyTMTC, with clear performance drops on challenging datasets such as ISCX-VPN-application and CICIoMT2024. Adding MoE to Mamba and Transformer improves the average F1 by 3.78% and 6.58%, respectively, showing that MoE helps strengthen the representation capability of the model.

We further compare the standard unidirectional Mamba with a bidirectional Mamba variant. As shown in Table IX, the unidirectional version achieves a higher average F1, increasing the score from 94.03% to 94.99%, and performs better on four of the five datasets. This suggests that preserving the original temporal order of traffic tokens is more suitable for HyTMTC than introducing an additional reverse scan. Therefore, the final model adopts the standard unidirectional Mamba.

Let the input of the encoder be $\mathbf{X} = [x_1, \ldots, x_L]$, where [CLS] (corresponding to x_1) is the target token. The ERF is defined as the sequence of gradient norms of the [CLS] representation h_1 in the encoder output with respect to the embeddings of each input token:

$$g_i = \left\| \frac{\partial h_1}{\partial x_i} \right\|_2, \quad i = 1, \ldots, L. \tag{14}$$

Figure 7 shows the normalized ERF heatmaps of 10 samples from the ISCX-VPN-application test set. Each row corresponds to one sample, and each column represents a token in the sequence. The color indicates the gradient sensitivity of each token to the target token. In model (b), only a small number of tokens exhibit strong gradient responses, while the remaining tokens show almost no gradient response. This indicates that Mamba can select important related tokens but has weak global interaction capability. Model (d) exhibits a wide coverage of effective gradient responses, but shows limited discrimination in gradient sensitivity across different tokens. After incorporating the MoE mechanism, model (c) shows improved global modeling compared with model (b), and model (e) demonstrates enhanced discrimination of token importance compared with model (d). This demonstrates that the MoE mechanism enhances sequence modeling performance. The ERF of HyTMTC exhibits both broader coverage of effective gradient responses and better discrimination of important tokens than the comparison models, indicating that the hybrid architecture simultaneously possesses global interaction capability and selective modeling capability in sequence modeling.

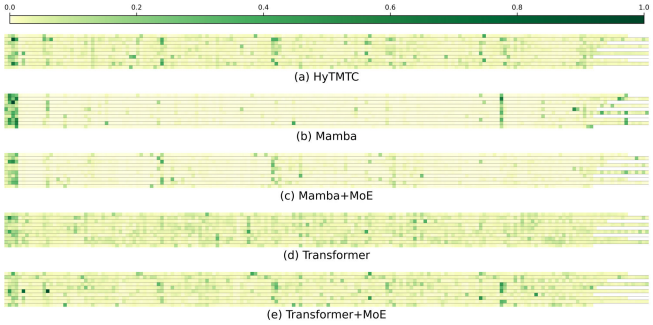


Fig. 7. Normalized ERF heatmaps of 10 samples from the ISCX-VPN-application test set. The color indicates the gradient sensitivity of different tokens to [CLS].

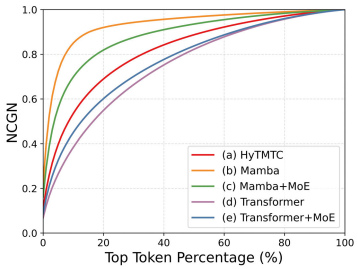


Fig. 8. Normalized cumulative ERF curves of HyTMTC and single-architecture models.

Figure 8 shows the normalized cumulative gradient norm (NCGN) curves of each model. The tokens are first sorted in descending order according to the gradient norm. The curve of model (b) rises rapidly at the initial stage, indicating that the gradient contribution is highly concentrated on a small number of tokens. The curve of model (d) rises the slowest, suggesting that the gradient contribution is more dispersed. The gradient accumulation rates of model (c) and model (e) lie between those of model (b) and model (d). The curve of HyTMTC lies in the middle among the five models, indicating that it achieves a balance between concentrated gradient contributions (selective modeling) and dispersed gradient contributions (global interaction).

F. Ablation Study

Table X presents the ablation study results for each component of HyTMTC. The training set contains 300 samples per class, while the validation and test sets remain unchanged, and the training is conducted for 10 epochs. In the following description, 'HyTMTC' refers to the full proposed method.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

TABLE X

ABLATION RESULTS OF DIFFERENT COMPONENTS IN HYTMTC. THE EXPERIMENTS ARE CONDUCTED BY PERFORMING ABLATION ON THE COMPLETE HYTMTC. IN THE 'TRAFFIC SEQUENCE' MODULE, 'W/' INDICATES THAT THE SEQUENCE IS CONSTRUCTED USING THE SPECIFIED INFORMATION. IN THE OTHER MODULES, 'W/' AND 'W/O' INDICATE USING OR NOT USING THE SPECIFIED MODULE, RESPECTIVELY

Component	Method		Dataset					Average F1
			MedBIoT	CICIoMT 2024	USTC -TFC	ISCX-VPN -service	ISCX-VPN -application	
Traffic Sequence	w/ bi-byte		0.9482	0.9121	0.9922	0.9500	0.7249	0.9055
	w/ bi-byte+length		0.9312	0.7662	**0.9978**	0.9739	0.7973	0.8933
	w/ bi-byte+direction		0.9424	0.9192	0.9877	0.9655	0.7228	0.9075
	w/ length		0.9532	0.5809	0.8667	0.9233	0.6792	0.8007
	w/ direction		0.6367	0.5792	0.4629	0.5226	0.3457	0.5094
	w/ length+direction		**1.0000**	0.5725	0.9029	0.8954	0.6950	0.8132
Embedding	w/o coarse+fine		0.9782	0.9056	0.9922	0.9676	0.8223	0.9332
	w/o coarse		0.9592	0.9065	0.9945	0.9740	0.8077	0.9284
	w/o fine		0.9391	0.9212	0.9889	0.9698	0.8238	0.9286
Self -Attention	w/o self-attention		0.9782	0.8141	0.9939	0.9753	0.7533	0.9030
Pretrained Parameters	30 per class	w/o	0.5752	0.3710	0.2975	0.3117	0.1925	0.3496
		w/	0.8409	0.9077	0.9413	0.7649	0.7098	0.8329
	100 per class	w/o	0.5442	0.4436	0.8105	0.7541	0.5385	0.6182
		w/	0.9424	0.9260	0.9853	0.9256	0.8525	0.9264
	300 per class	w/o	0.6515	0.7319	0.9598	0.9495	0.7049	0.7995
Parameters Sharing	w/o		0.9758	0.9176	0.9695	0.9509	0.8202	0.9268
	HyTMTC		0.9782	**0.9260**	0.9923	**0.9800**	**0.8733**	**0.9499**

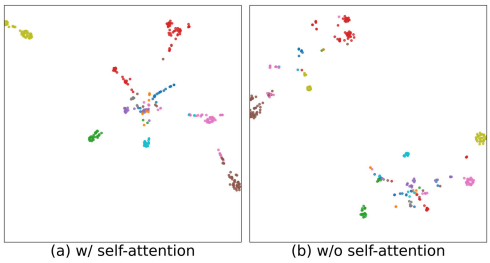


Fig. 9. The t-SNE visualization of the encoder outputs on the ISCX-VPN-application test set.

(1) Traffic Sequence: The combinations using byte information achieve an average F1 of about 90%, which is approximately 5% lower than HyTMTC. In the combinations using only transmission pattern information, those including length information show an average F1 about 15% lower than HyTMTC, while the combination using only direction information shows no classification ability. This indicates that byte information contributes primarily to representing traffic. Meanwhile, adding transmission pattern information on top of byte information can further improve the model's ability to represent traffic data.

(2) Embedding: Using both the coarse semantic and the fine semantic embedding layers improves the F1 by 1.61% compared with not using these two layers. When only one of the two layers is used, the F1 decreases slightly. Therefore, HyTMTC adopts both layers simultaneously. This demonstrates that injecting network traffic-specific structural semantic information into the sequence enhances the model's representation capability.

(3) Self-Attention Mechanism: Introducing the self-attention mechanism during the fine-tuning stage improves the F1 of HyTMTC by 11.19% on the complex IoT scenario (CICIoMT2024) and by 12% on the VPN tunnels encryption scenario (ISCX-VPN-application). Fig. 9 illustrates the contribution of the self-attention mechanism to modeling the ISCX-VPN-application test set. In (a) 'w/ self-attention', samples of the same class are more compact and the separation between different classes is larger, whereas in (b) 'w/o self-attention' the class separability is weaker. This indicates that the self-attention mechanism contributes to the enhancement of model representation in complex traffic scenarios.

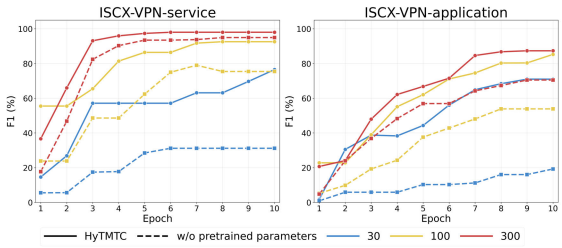


Fig. 10. The growth trend of F1 with increasing training epoch for HyTMTC and the model without pretrained parameters on ISCX-VPN-service and ISCX-VPN-application datasets.

TABLE XI

ABLATION STUDY ON IP ADDRESS AND PORT INFORMATION

IP/port	Dataset				Average F1
	MedBIoT	CICIoMT2024	NUDT	CSTNET	
mask	0.9592	0.9204	0.9161	0.9673	0.9408
random	0.9782	0.9260	0.9117	0.9677	**0.9459**

(4) Pretrained Parameters: Under three training sample settings, HyTMTC achieves significantly higher F1 than the model without pretrained parameters. When the number of training samples is 30 or 100, HyTMTC demonstrates few-shot learning capability, whereas the model without pretrained parameters achieves average F1 of only 34.96% and 61.82%. Fig. 10 shows that as the number of training epochs increases, HyTMTC converges faster and reaches effective classification performance after 10 epochs. In contrast, the model without pretrained parameter converges more slowly and still fails to achieve classification capability after 10 epochs under few-shot conditions. These results demonstrate the effectiveness of using the MLM task for learning general knowledge from network traffic.

(5) Parameter Sharing: In HyTMTC, the Mamba-MoE layers share one set of parameters. Compared with the non-sharing variant, this setting improves the average F1 from 92.68% to 94.99%. The results suggest that parameter sharing between the two Mamba-MoE layers does not compromise classification performance. Meanwhile, it reduces the number of trainable parameters from 14.49M to 8.20M, while keeping the layer-wise computation unchanged.

(6) IP/port: Table XI shows the ablation study results for IP addresses and port numbers. 'Random' means that the IP addresses and ports in each bidirectional flow are randomized in a flow-consistent manner, which is the data processing strategy adopted by HyTMTC. 'Mask' means that the tokens corresponding to IP addresses and ports in the multi-modal traffic representation are replaced with [MASK]. The results show that the two settings lead to very similar performance. This suggests that the randomization strategy does not retain the original IP/port identities or provide the model with dataset-specific shortcuts. The results under the 'Mask' setting further indicate that HyTMTC does not make predictions based on IP addresses or ports, but relies more on packet bytes, packet length, direction, and flow-level behavior.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

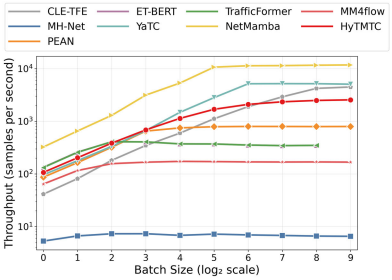


Fig. 11. The comparison of throughput between HyTMTC and the baselines.

TABLE XII
THE NUMBER OF PARAMETERS AND FLOPS OF HYTMTC AND THE BASELINES

Methods	Params	FLOPs@BS=1
FlowPrint [15]	-	-
CLE-TFE [28]	44.85M	1.15G
MH-Net [31]	194.29M	4.74G
PEAN [7]	1.69M	1.60G
ET-BERT [5]	85.62M	27.18G
YaTC [9]	1.78M	905.96M
TrafficFormer [6]	85.62M	27.18G
NetMamba [8]	5.91K	414.21K
MM4flow [39]	170.13M	65.29G
HyTMTC	8.20M	957.16M

G. Inference Efficiency Comparison

We report the number of parameters, FLOPs, and throughput of each model. All experiments are conducted on one NVIDIA GeForce RTX 4090 GPU. As shown in Table XII, HyTMTC has 8.20M parameters and 957.16M FLOPs at batch size 1, which is much smaller than several large pre-training baselines such as ET-BERT, TrafficFormer, and MM4flow. Although HyTMTC is not the smallest model, it achieves the best overall classification performance with a moderate model size and computational cost. Figure 11 shows the throughput of different models under varying batch sizes. The throughput of HyTMTC stays in the middle range among the compared methods when the batch size is larger than 64, and becomes relatively stable when the batch size exceeds 128. This indicates that HyTMTC does not pursue the highest inference speed, but provides a reasonable efficiency-performance trade-off. Since HyTMTC is mainly designed for offline multi-scenario traffic classification rather than real-time lightweight detection, this efficiency level is acceptable. In future work, we will further study how to improve inference efficiency while preserving the representation capability of HyTMTC.

V. LIMITATIONS

Although HyTMTC performs consistently across different traffic classification scenarios, several practical issues remain worth noting.

First, as a pre-training-based model, HyTMTC requires extra computation during pre-training, and its inference cost is higher than that of lightweight statistical classifiers. To keep the cost manageable, HyTMTC uses only the first few packets of each flow and adopts a compact traffic representation. For high-throughput deployment, further optimization such as model compression or quantization could be considered.

Second, class imbalance is common in real traffic, where some applications or attack types may occur much less frequently than others. HyTMTC does not specifically target imbalance-aware learning, such as class reweighting or resampling. Even so, the few-shot results show that HyTMTC can learn useful representations from limited labeled samples, suggesting that its pre-trained multi-modal representation reduces the need for abundant samples in each class to some extent. In highly skewed deployment settings, imbalance-aware training could further improve its robustness.

Finally, real network environments are more dynamic than offline datasets. Traffic patterns may change with new applications, protocol updates, encryption mechanisms, and emerging attacks. Although HyTMTC shows good generalization in our experiments, long-term deployment would still benefit from continuous model updates and drift monitoring. These issues point to practical extensions of HyTMTC rather than limitations of the core method.

VI. CONCLUSION

This paper proposes HyTMTC, a multi-scenario network traffic classification method based on the pre-training–fine-tuning paradigm. HyTMTC is built on a hybrid Transformer-Mamba architecture, which provides both global interaction capability and selective modeling capability for sequence representation. Based on the characteristics of network traffic, HyTMTC designs a multimodal traffic sequence by injecting semantic and structural information into the model simultaneously. In addition, HyTMTC designs a self-attention mechanism during the fine-tuning stage to fuse multimodal information, thereby improving classification performance in complex scenarios. HyTMTC demonstrates effective classification performance and stability across multiple network traffic scenarios, outperforming nine baselines. HyTMTC also demonstrates effective classification capability in complex or few-shot scenarios, and maintains detection ability for unknown attacks. In future work, we will further improve the deployment efficiency of HyTMTC and explore more practical adaptation strategies for real network environments.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, Hum. Lang. Technol., vol. 1, Jun. 2019, pp. 4171–4186.

[2] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[3] A. El-Nouby et al., "Scalable pre-training of large autoregressive image models," 2024, *arXiv:2401.08541*.

[4] Q. Sun et al., "Generative multimodal models are in-context learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 14398–14409.

[5] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[6] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.

[7] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.

[8] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–11.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

[9] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[10] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 4898–4906.

[11] M. Al-Hisnawi and M. Ahmadi, "Deep packet inspection using quotient filter," *IEEE Commun. Lett.*, vol. 20, no. 11, pp. 2217–2220, Nov. 2016.

[12] P. Khandait, N. Hubballi, and B. Mazumdar, "Efficient keyword matching for deep packet inspection based network traffic classification," in *Proc. Int. Conf. Commun. Syst. Netw. (COMSNETS)*, Jan. 2020, pp. 567–570.

[13] R. T. El-Maghraby, N. M. Abd Elazim, and A. M. Bahaa-Eldin, "A survey on deep packet inspection," in *Proc. 12th Int. Conf. Comput. Eng. Syst. (ICCES)*, Dec. 2017, pp. 188–197.

[14] A. Azab, M. Khasawneh, S. Alrabaee, K.-K.-R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 676–692, Jun. 2024.

[15] T. Van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 27, 2020, pp. 1–18.

[16] S. Dong, "Multi class SVM algorithm with active learning for network traffic classification," *Expert Syst. Appl.*, vol. 176, Aug. 2021, Art. no. 114885.

[17] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.

[18] O. Belkadi, A. Vulpe, Y. Laaziz, and S. Halunga, "ML-based traffic classification in an SDN-enabled cloud environment," *Electronics*, vol. 12, no. 2, p. 269, Jan. 2023.

[19] J. Kotak, I. Yankelev, I. Bibi, Y. Elovici, and A. Shabtai, "VPN-encrypted network traffic classification using a time-series approach," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 2, pp. 2225–2242, Apr. 2025.

[20] I. A. Alwhbi, C. C. Zou, and R. N. Alharbi, "Encrypted network traffic analysis and classification utilizing machine learning," *Sensors*, vol. 24, no. 11, p. 3509, May 2024.

[21] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 397–405.

[22] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep. 2022.

[23] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[24] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.

[25] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.

[26] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, pp. 1–18, 2023, doi: 10.1109/TDSC.2023.3242134.

[27] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

[28] H. Zhang, X. Xiao, L. Yu, Q. Li, Z. Ling, and Y. Zhang, "One train for two tasks: An encrypted traffic classification framework using supervised contrastive learning," 2024, *arXiv:2402.07501*.

[29] X. Deng, Q. Li, and K. Xu, "Robust and reliable early-stage website fingerprinting attacks via spatial–temporal distribution analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 1997–2011.

[30] X. Xiao et al., "RBLJAN: Robust byte-label joint attention network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2161–2178, May 2025.

[31] H. Zhang et al., "Revolutionizing encrypted traffic classification with MH-net: A multi-view heterogeneous graph model," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 1, pp. 1048–1056.

[32] G. Li, Y. Gao, J. Ren, and S. Chen, "BPF-GNN: A multi-granularity feature extraction model using graph neural networks for encrypted traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 23, pp. 3105–3117, 2026.

[33] X. Zhao, M. He, and X. Wang, "Semi-supervised learning with interpolation and pseudo-labeling for few-label intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 5, pp. 4302–4320, Oct. 2025.

[34] X. Yao, F. Liu, J. Jiang, G. Zhong, and C. L. Philip Chen, "Category-constrained broad recurrent system for cloud anomaly detection," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 5, pp. 5015–5030, Oct. 2025.

[35] X.-Y. Chen, L. Han, D.-C. Zhan, and H.-J. Ye, "MIETT: Multi-instance encrypted traffic transformer for encrypted traffic classification," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 15, pp. 15922–15929.

[36] V. Tong et al., "Encrypted traffic classification through deep domain adaptation network with smooth characteristic function," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 1, pp. 331–343, Feb. 2025.

[37] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.

[38] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.

[39] L. Yang et al., "MM4flow: A pre-trained multi-modal model for versatile network traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 1664–1678.

[40] Y. Zheng, X. Lian, Z. Dang, C. Peng, C. Yang, and J. Ma, "A semi-supervised anomaly network traffic detection framework via multimodal traffic information fusion," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 4455–4459.

[41] X. Lian, Y. Liu, S. Wang, and Y. Zheng, "Payload level anomaly network traffic detection via semi-supervised contrastive learning," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2024, pp. 2559–2566.

[42] X. Lian, Y. Zheng, Z. Dang, C. Peng, and X. Gao, "Semi-supervised anomaly traffic detection via multi-frequency reconstruction," *Pattern Recognit.*, vol. 161, May 2025, Art. no. 111215.

[43] X. Lian, C. Cao, T. Zhong, Y. Wang, K. Chen, and F. Zhou, "Decompose to understand, fuse to detect: Frequency-decoupled anomaly detection for encrypted network traffic," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun.*, May 2026, pp. 1–10.

[44] Z. Dang, Y. Zheng, X. Lian, C. Peng, Q. Chen, and X. Gao, "Semi-supervised learning for anomaly traffic detection via bidirectional normalizing flows," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 5, pp. 5106–5117, Oct. 2025.

[45] X. Lian, C. Cao, Y. Liu, X. Xu, Y. Zheng, and F. Zhou, "Facing anomalies head-on: Network traffic anomaly detection via uncertainty-inspired inter-sample differences," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 3908–3917.

[46] X. Lian, Y. Zheng, Y. Liu, F. Zhou, C. Peng, and X. Gao, "Contextual masking distillation for network traffic anomaly detection," *IEEE Trans. Inf. Forensics Security*, vol. 21, pp. 1273–1286, 2026.

[47] X. Wang, Y. Wang, Y. Lai, Z. Hao, and A. X. Liu, "Reliable open-set network traffic classification," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2313–2328, 2025.

[48] G. Bovenzi et al., "Benchmarking class incremental learning in deep learning traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 1, pp. 51–69, Feb. 2024.

[49] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[50] X. Lian, C. Cao, T. Zhong, and F. Zhou, "Disentangling multi-view scanning in mamba for network traffic anomaly detection," 2026, *arXiv:2607.22829*.

[51] L. Zhang and Y. Wan, "TranMamba: A lightweight hybrid transformer-mamba network for single image super-resolution," *Signal, Image Video Process.*, vol. 19, no. 5, pp. 1–12, May 2025.

[52] C. Liu, D. Zhang, G. Lu, W. Yin, J. Wang, and G. Luo, "SRMamba-T: Exploring the hybrid mamba-transformer network for single image super-resolution," *Neurocomputing*, vol. 624, Apr. 2025, Art. no. 129488.

[53] Y. He, B. Tu, B. Liu, J. Li, and A. Plaza, "HSI-MFormer: Integrating mamba and transformer experts for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5621916, doi: 10.1109/TGRS.2025.3564167.

[54] X. Zhu, Q. Ruan, S. Qian, and M. Zhang, "A hybrid model based on transformer and mamba for enhanced sequence modeling," *Sci. Rep.*, vol. 15, no. 1, p. 11428, Apr. 2025.

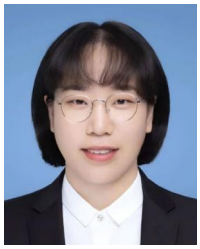
[55] B. Lenz et al., "Jamba: Hybrid transformer-mamba language models," in *Proc. 13th Int. Conf. Learn. Represent.*, 2025, pp. 1–17.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.

[56] Z. Shao, Z. Wang, X. Yao, M. G. H. Bell, and J. Gao, "ST-MambaSync: Complement the power of mamba and transformer fusion for less computational cost in spatial–temporal traffic forecasting," *Inf. Fusion*, vol. 117, May 2025, Art. no. 102872.
[57] X. Zhang, W. Ou, X. Wu, and C. Zhang, "MHS-VIT: Mamba hybrid self-attention vision transformers for traffic image detection," *PLoS ONE*, vol. 20, no. 6, Jun. 2025, Art. no. e0325962.
[58] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surveys*, vol. 54, no. 6, pp. 1–35, Jul. 2022.
[59] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 12381–12392.
[60] X. Han et al., "Pre-trained models: Past, present and future," *AI Open*, vol. 2, pp. 225–250, Apr. 2021.
[61] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, May 2016, pp. 407–414.
[62] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark Android malware datasets and classification," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Montreal, QC, Canada, Oct. 2018, pp. 1–7.
[63] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.
[64] A. Guerra-Manzanares, J. Medina-Galindo, H. Bahsi, and S. Nõmm, "MedBIoT: Generation of an IoT botnet dataset in a medium-sized IoT network," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Jul. 2020, pp. 207–218.
[65] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, "CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT," *Internet Things*, vol. 28, Dec. 2024, Art. no. 101351, doi: 10.1016/j.iot.2024.101351.
[66] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.
[67] S. Zhao, S. Chen, F. Wang, Z. Wei, J. Zhong, and J. Liang, "A large-scale mobile traffic dataset for mobile application identification," *Comput. J.*, vol. 67, no. 4, pp. 1501–1513, Apr. 2024.
[68] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu. (2022). *CSTNET-TLS 1.3 Dataset*. [Online]. Available: https://drive.google.com/drive/folders/1BUo5TMRuXNvTqNYy0RLeHk4l4Q3BuzSk
[69] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent.*, San Diego, CA, USA, 2015, pp. 1–15.



Yuanming Huang received the B.E. degree in electronic science and technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2021, where she is currently pursuing the Ph.D. degree with the School of Integrated Circuits. Her research interests include cybersecurity and deep learning.



Xiaojuan Wang (Member, IEEE) received the Ph.D. degree in electronic science and technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2015. She is currently an Associate Professor with the School of Cyberspace Security. Her research interests include deep learning, complex networks, and human gesture recognition.



Mingshu He (Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2022. He is currently doing research with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. His research interests include network security, anomaly detection, and machine learning.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:19 UTC from IEEE Xplore. Restrictions apply.